DC Office Room-Level Smart Energy Management System

Project Proposal: IoT Room Monitoring, Potential Energy-Waste Detection, Controlled Automation, and a Self-Hosted Data, Analysis and Dashboard Platform

Project Team

Design-stage document

Abstract

This proposal describes a prototype design for one representative DC Office room. The existing ESP8266 (NodeMCU) prototype reads motion, light, temperature and humidity, applies transparent local rules, and publishes data over MQTT to a self-hosted server. The server stores readings and events in a database, computes operational analysis, and serves a web dashboard for status, alerts, manual control and reports. This is a proposed design and test plan, not a deployment report. It does not claim completed implementation, real operational measurements, verified energy savings or measured kWh reduction. Building-wide power systems are outside the project scope.

1 Executive Summary and Problem

The proposed system explores how a low-cost IoT prototype could help manage appliance operation in one room. It combines sensor readings, a local rule-based controller, device-state and event tracking, a custom storage and analysis back end, a web dashboard, and a cautious control policy. The first goal is to detect and communicate *potential* energy waste; automation is secondary and must pass safety and reliability checks. A self-hosted stack replaces the earlier Blynk plan so that the project controls what is stored, analysed and displayed.

Problem being investigated. Lights or fans may remain on when a room appears vacant, or a light may remain on when daylight is sufficient. Users may also lack a record of switching events and run time. These are hypotheses to test, not established findings about an actual DC Office. Motion sensors can miss stationary occupants, and a relay command alone cannot confirm that an appliance physically switched or how much electricity it consumed.

Objectives.

1. Reuse the existing prototype for a single-room DC Office use case.
2. Monitor motion, ambient-light indication, temperature and humidity.
3. Track commanded device states, operating duration and relevant events.
4. Identify potential vacancy- and daylight-related waste using transparent rules.
5. Store readings, states and events persistently in a local database.
6. Provide a web dashboard for live status, alerts, event history, manual control and analysis.
7. Produce descriptive analysis (runtime, alert counts, data quality) with clearly labelled limits.
8. Enable only approved automatic actions after calibration, testing and safety checks.

2 Scope, Components and System Architecture

In scope.

- One room represented by the current prototype, not a building or multi-room deployment.
- PIR motion, LDR light level and DHT11 temperature/humidity readings.
- Light, fan, AC and PC as device categories in the interface and control policy.
- AUTO/MANUAL mode, status display, alerts, event history and runtime estimates based on commanded states.
- A local MQTT broker, back-end service, database and web dashboard on the office network.
- Configurable rules, calibration, manual override and planned functional testing.

Out of scope for the initial prototype. Building-wide deployment, verified bill reduction, direct kWh measurement, guaranteed occupancy detection, advanced AI claims, internet-exposed access, and automatic forced shutdown of AC/PC. These would require additional evidence, equipment or approval.

Hardware components.

ESP8266 / NodeMCU	Reads sensors, runs local rule logic, publishes data and receives commands.
PIR sensor	Indicates detected motion; it does not reliably prove a room is empty.
LDR	Relative light reading; the threshold must be calibrated in the room.
DHT11	Approximate temperature and humidity; accuracy and response are limited.
Relay module	Switches supported loads only when ratings, wiring, isolation and installation are suitable.
Server host	A PC or small single-board computer on the office LAN (to be confirmed) running the broker, service and database.

Software stack and languages.

Layer	Language / tool	Purpose
Firmware	C++ (Arduino framework)	Sensor reading, rules, safety gate, MQTT client; libraries such as ESP8266WiFi, PubSubClient, ArduinoJson and a DHT library.
Messaging	MQTT, JSON	Lightweight two-way transport; Eclipse Mosquitto as broker.
Back end	Python 3, FastAPI	MQTT subscriber, REST API, command relay, analysis functions.
Storage	SQLite, SQL	Single-file relational database for readings, states, events and runtime sessions.
Dashboard	HTML, CSS, JavaScript, Chart.js	Browser interface served by the back end; Chart.js hosted locally so it works without internet.

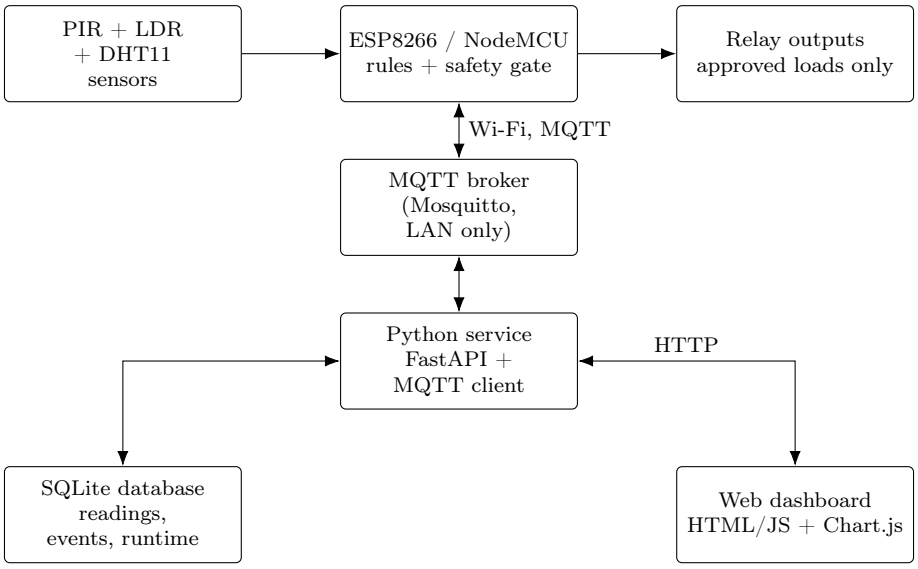


Figure 1: High-level architecture (proposed logical design).

The controller keeps its safety and rule behaviour locally. The server is used for viewing, storage, analysis and commands; it must not be the only safety mechanism.

3 Operating Workflow and Functional Features

Typical control cycle on the ESP8266.

1. Read sensors and the current control mode; attach a timestamp.
2. Check sensor validity, stability and recent manual override.
3. If AUTO mode and a rule condition are satisfied, evaluate the device-specific safety policy. Otherwise keep the current state or follow the manual command and log the event.

4. Apply the permitted action or raise an alert; publish the new state and event.

Functional features.

Live status	Sensor values, AUTO/MANUAL mode, commanded ON/OFF state, last-update time. Stale or invalid readings are marked rather than used for unsafe decisions.
Potential-waste alerts	Prolonged no-motion while a device is commanded ON, or a light commanded ON when calibrated daylight appears sufficient.
Manual override	An authorised user changes mode or device command; a recent manual ON prevents immediate automatic OFF.
Event history	Timestamp, context, rule triggered, requested action, result and reason, filterable on the dashboard.
Runtime indicator	Estimated from commanded relay state and timestamps; labelled as an estimate, not measured energy.
Connectivity handling	Online/offline status and communication loss are shown; local safety rules do not depend on the server.

Modes. AUTO: configured rules may initiate only actions permitted by the safety policy. **MANUAL:** user commands take priority and automation must not reverse them unexpectedly. A clear mode indicator and a way to return to a safe state are required. If a sensor fails or readings become stale, automatic OFF decisions are suppressed and an alert is raised where possible.

4 Control Rules and Device Safety Policy

The initial intelligence is explainable, configurable, rule-based control. It is not presented as a trained AI model. Thresholds are starting examples for testing, not final values; they are tuned only after observing the prototype and room conditions.

Vacancy-related rule (example stages).

1. No PIR motion for 5 minutes: mark a *vacancy candidate*.
2. No motion for 10 minutes: issue a warning or pending-action notification.
3. No motion for 15 minutes: consider automatic OFF only for an approved device, if the sensor is valid, AUTO mode is active, no manual override is recent and minimum switching intervals are satisfied.

PIR detects motion, not people. A stationary person may not trigger it, so timers must be configurable and validated; alert-first behaviour is safer during early testing.

Daylight-related light rule. After LDR calibration, if ambient light stays above the configured threshold for a confirmation period (example: 3 minutes), the system may turn a light OFF in AUTO mode. Separate ON and OFF thresholds (hysteresis), valid readings and a minimum interval between changes prevent rapid relay toggling. This behaviour stays disabled until calibration and testing are complete.

Device policy matrix.

Device	Initial policy	Reason / condition
Light	Alert + possible AUTO OFF	Only after LDR calibration or validated vacancy rule, stable data, confirmation time and override checks.
Fan	Alert + possible AUTO OFF	Only after vacancy logic is tested and the fan's use and safety requirements are reviewed.
AC	Monitor / alert only	No forced power cycling; use an appropriate AC control interface and approval before considering automation.
PC	Monitor / alert only	No automatic power cut; abrupt disconnection can cause data loss or equipment problems.

Safety checks before an action.

- Sensor readings must be valid, stable and recent; otherwise automatic switching is blocked.
- Confirm AUTO mode, device eligibility, no active manual override and a minimum switching interval.
- Commands arriving from the server or dashboard pass the same safety gate in the firmware; the server cannot bypass it.

- Use a relay and enclosure with suitable ratings, isolation and wiring, installed by qualified personnel. Never switch mains wiring on an exposed prototype board.
- Log the rule, command and outcome separately. Relay command state is not proof of physical load state.

The gate can be expressed compactly in the firmware (illustrative only):

```
bool autoOffAllowed(const Device& d) {
  return mode == AUTO && d.autoEligible && sensorsValid()
      && (millis() - lastManualCmd) > MANUAL_HOLD_MS
      && (millis() - d.lastSwitch)  > MIN_SWITCH_MS;
}  // autoEligible is false for AC and PC
```

5 Firmware and Communication Design

Timekeeping. The ESP8266 has no battery-backed clock, so the firmware gets time through NTP and sends a device timestamp with each message; the server also stores its receive time. Readings taken before time sync are flagged, not given a guessed time.

MQTT topics.

Topic	Direction	Content
`dcems/room1/sensors`	ESP → server	Periodic sensor snapshot (JSON).
`dcems/room1/state`	ESP → server	Mode and commanded device states, on change (retained).
`dcems/room1/events`	ESP → server	Rule, alert, override and fault events.
`dcems/room1/cmd`	server → ESP	Mode change or device command.
`dcems/room1/ack`	ESP → server	Command result: accepted, or blocked with reason.
`dcems/room1/status`	ESP → server	Online/offline, using MQTT Last Will and a periodic heartbeat.

Example messages.

```
{"ts":1761800000,"pir":0,"ldr":540,"temp":29.5,"hum":68,"valid":true}
{"ts":1761800300,"type":"vacancy_candidate","device":"light",
 "rule":"V1","decision":"alert","reason":"no motion 5 min"}
```

Publishing policy (example values). Sensor snapshots every 10–30 s; state and events only on change; heartbeat about every 30 s. A command is not successful until an `ack` arrives; the dashboard shows sent, accepted or blocked.

Server or network loss. The ESP keeps running local rules and holds unsent events in a bounded queue for sending after reconnection. On overflow a `data_gap` event is recorded; missing data is never stored as zero.

6 Data Storage Design

SQLite is chosen for the prototype because it needs no separate database server and keeps everything in one file that is easy to back up and inspect; the schema can move to PostgreSQL if volume grows.

Tables.

`sensor_readings`	Device and server timestamps, PIR, LDR, temperature, humidity, validity flag.
`device_states`	Timestamp, device, commanded ON/OFF, control mode, source (auto, manual, server).
`events`	Timestamp, event type, device, mode, rule ID, proposed action, decision (allowed, blocked, alert), reason, context.
`runtime_sessions`	Device, start, end, end-known flag; estimated duration from commanded state.
`config`	Timers and thresholds with change history.
`audit_log`	Dashboard logins and commands sent.

Data handling rules. Missing or invalid readings are stored with a validity flag or as NULL, never as zero. A runtime session with an unknown end (restart, outage) is marked end-unknown and reported separately. Raw readings are kept for a defined period (to be decided, e.g. 90 days) and may then be reduced to hourly aggregates; events and sessions are kept longer. The database file is copied nightly to a separate location, and a restore is tested once during evaluation.

7 Server, Dashboard and Security

Back-end API (FastAPI).

Endpoint	Purpose
`GET /api/live`	Latest readings, mode, device states, online/offline.
`GET /api/readings`	Sensor history for charts (time-range query).
`GET /api/events`	Filterable events by type, device and time.
`GET /api/analysis/runtime`	Estimated daily runtime per device.
`GET /api/analysis/summary`	Alert, override, blocked-action and fault counts.
`POST /api/cmd`	Mode or device command (authentication required); waits for `ack`.
`GET /api/export/events.csv`	CSV export for test reports.

Dashboard pages.

1. **Live:** sensor cards, mode, device states, last-update time, and a stale/offline banner.
2. **Control:** AUTO/MANUAL and device switches, with command status (sent, accepted, blocked).
3. **Alerts and Events:** current alerts and a filterable event history table.
4. **Analysis:** runtime bar charts, LDR and temperature line charts, and rule-outcome counts.
5. **Settings:** view and change timers and thresholds; changes are logged and sent to the ESP.

The first version polls the API every few seconds. Every runtime figure is labelled "estimate based on commanded state".

Security. The broker and dashboard stay on the local network and are not exposed to the internet. MQTT requires a username and password (TLS is considered on a shared network). Control and settings require dashboard login; commands and logins are written to the audit log. The server validates input and the firmware re-checks every command against the safety gate. Credentials are kept outside the source repository.

8 Analysis Methods

The analysis is descriptive. It uses SQL aggregation and interval calculations in Python; it does not use machine learning, and no model accuracy is claimed. Statistical or anomaly detection could be explored later, only after sufficient reliable data exists.

Indicator	Definition and use
Estimated runtime	Sum of ON-to-OFF durations per device and day from commanded states; sessions with unknown end are reported separately.
Potential-waste time	Time a device is commanded ON while a vacancy candidate or sufficient-daylight condition holds (overlap of intervals). Indicates possible waste, not wasted energy.
Rule outcomes	Counts per rule and device of alerts, allowed actions and blocked actions, grouped by reason.
Override after auto-OFF	Manual ON within a set time after an automatic OFF; a possible indicator of false vacancy or daylight decisions.
Data quality	Received versus expected readings, share of invalid or stale readings, offline periods and data gaps.
Calibration profiles	Hourly distributions of LDR and PIR activity to help choose daylight thresholds and vacancy timers.

Measurement boundary. No energy meter is in the current hardware list, so the prototype cannot report verified watts, kWh, cost reduction or percentage savings. Runtime and event counts are operational indicators only. Quantifying energy impact later needs an energy meter, a baseline, and comparison of equivalent periods while accounting for occupancy, weather, schedules and manual behaviour.

9 Verification and Evaluation Plan

Testing begins with low-voltage simulation or safe bench testing. Mains-connected loads are tested only with suitable rated hardware, enclosure, isolation and qualified supervision. Actual outcomes are recorded rather than assumed.

Table 9: Planned test cases.

ID	Test	Procedure	Expected result
T1	Sensor readings	Observe PIR, LDR and DHT11 in normal conditions.	Sensor values update; faults and stale data are clearly indicated.
T2	Vacancy stages	Keep a device commanded ON; simulate no motion.	Vacancy candidate and warning appear; only eligible devices reach AUTO OFF logic.
T3	Stationary occupant	Stay still during vacancy timing.	PIR limitation documented; timers tuned or alert-first behaviour kept.
T4	Daylight threshold	Test LDR in dim and bright light after calibration.	Light-OFF condition requires a stable threshold and the confirmation time.
T5	Hysteresis	Vary light around the threshold.	No rapid ON/OFF cycling of the relay.
T6	Manual override	Turn a device ON during an automation window.	The recent manual override is respected; no automatic OFF.
T7	AC/PC restriction	Trigger vacancy rules with AC/PC selected.	Alert only; no automatic power cut.
T8	Sensor/network fault	Disconnect a sensor or the network link.	Unsafe automatic OFF is suppressed and the fault is recorded.
T9	Event/runtime log	Toggle commands; inspect timestamps.	Events and runtime are consistent and labelled as estimates.
T10	Restart/recovery	Restart the controller during operation.	Predictable startup, safe outputs, and status recovers.
T11	Server outage	Stop broker or service, generate events, restore.	Local rules continue; queued events arrive; gaps are recorded.
T12	Storage integrity	Compare database rows with ESP serial log.	Counts and timestamps agree within the stated tolerance.
T13	Command path	Send dashboard commands, including AC/PC power-off.	Allowed commands are acknowledged; blocked ones show the reason.
T14	Analysis accuracy	Recalculate runtime and counts by hand.	Dashboard figures match the hand calculation.
T15	Access control	Send a command without login or with wrong MQTT credentials.	Request is rejected and recorded.

Evaluation indicators. Sensor update reliability and invalid/stale frequency; correct triggering of vacancy and daylight rules; number of false or inappropriate automation decisions; manual-override success and absence of repeated switching; completeness of stored events and consistency of runtime estimates; dashboard accuracy against source data; and safe blocking of AC/PC power-disconnection commands from every path.

Acceptance approach. Pass/fail criteria are defined before each test. A feature is not validated just because a command appears on the dashboard; controller behaviour, relay output, stored event and, where safe, the physical load response are inspected. Limitations and failed tests are reported with the successes.

10 Risks, Limitations, Roadmap and Conclusion

Key risks and mitigations.

False vacancy detection	Conservative configurable timers, alert-first trials, manual override, and recognition that PIR may miss stationary occupants.
Incorrect daylight decision	Calibrate the LDR in the room, use confirmation and hysteresis, disable the rule on invalid readings.
Unsafe switching	Limit automation by device category; use rated, enclosed hardware and qualified installation.
Misleading energy claims	Separate commanded state and runtime from physical state and measured energy; no kWh or savings without metering.
Server, broker or network failure	Local rules on the ESP, bounded event queue, safe startup state, recorded gaps.

Unauthorised control	LAN-only access, authentication, audit log, firmware safety gate.

Proposed development stages.

1. **Stage 1: Inspect and document.** Confirm wiring, relay ratings and features; choose the server host.
2. **Stage 2: Monitor only.** Set up broker, database and service; send sensor data with Last Will; build the Live page; validate readings and timestamps.
3. **Stage 3: Events and calibration.** Add alert-only rules, event history and the Alerts page; use the stored data to calibrate vacancy timers and LDR thresholds.
4. **Stage 4: Limited automation.** Add the command and acknowledgement path and Control page; enable only approved light and fan actions after passing the tests; AC and PC stay alert-only.
5. **Stage 5: Analyse and evaluate.** Complete the Analysis page and CSV export, run T1–T15, test backup and restore, and report results.

Conclusion. This proposal adapts the existing ESP8266 room prototype to a single DC Office room and adds a self-hosted platform for data storage, analysis and a web dashboard. It prioritises safety, manual control and transparent, checkable records over aggressive switching or unsupported AI claims. Whether the approach reduces energy use remains open until the prototype is tested and, for quantitative claims, energy is actually measured.